信息检索导论

Fundamental of Information Retrieval

第2讲 词项词典及倒排记录表

The term vocabulary and postings lists

提纲

1. **文档**
2. **词项**
    1. 通常做法+非英语处理
    2. 英语
3. **跳表指针**
4. **短语查询**

本讲的内容

- **索引构建过程(特别是预处理)**
- **如何对索引文档进行处理得到词典**
  - **理解文档(document)的概念**
  - **词条化(Tokenization)，理解词条(token)的概念**
  - **词项生成，理解词项(term)的概念**
- **倒排记录表**
  - **更快的合并算法: 跳表法(skip list)**
  - **短语查询的处理及带位置信息的倒排索引**

提纲

1. 上一讲回顾
2. **文档**
3. 词项
   - 通常做法+非英语处理
   - 英语
4. 跳表指针
5. 短语查询

文档分析

- **文档格式处理**
  - pdf/word/excel/html?



- **文档语言识别**

- **文档编码识别**

回顾
倒排索引构建



待索引文档

Friends, Romans, countrymen.

Tokenizer 词条化工具

词条流

More on these later.

Linguistic modules 语言分析工具

Friends Romans Countrymen

修改后的词条

friend roman countryman

Indexer

倒排索引

friend → 2 → 4

roman → 1 → 2

countryman → 13 → 16

多格式/语言并存

- **文档语言识别**
  - 待索引**文档集**可能同时包含多种语言的文档
  - 在同一索引中**词汇表**中包含来自**多个语言**的词项
  - 有时**文档或者其部件**中包含多种语言/格式
    - 法语**正文**邮件中附带德语pdf格式附件

文档编码识别



- 作为索引构建过程的输入，数字文档一般由文件中或者Web服务器上的一系列**字节**组成
- 因此，**文档处理的第一步往往是将这些字节序列转换成线性的字符序列**
- 对于ASCII编码的纯英文文本，相对容易，但实际中非常复杂
  - 字符序列可能采用各种单字节或者多字节编码方式（比如 Unicode中的UTF-8 编码）
  - 也可能采用不同国家、不同厂家的特定编码方式
- 因此，为了实现从字节序列到字符序列的转换，首先要正确地判断出文档的编码方式
- **假设文档只由一系列的文本字符构成**。由于用户只想和数据的本来面目打交道，所以商业产品往往都需要支持各种文档和编码格式。
- 在应用中用户常常把**文档**想成就是**文本**，甚至都不会意识到它们在存放时还有编码格式问题

多格式/语言并存

- 需要确定索引的**文档单位（document unit)**
- 至今为止，我们都假设在建立索引时每篇文档就是固定的索引单位
- 比如，将某个目录下的每个文件都看成一个文档。但是，很多情况下并非如此
  - 传统的UNIX（mbox格式）**邮件系统**将某个邮件目录下的一系列邮件都存放到单个文件中，但是我们希望将**每封邮件都看成一个独立的文档**
  - 很多邮件还有附件，我们还希望将每封邮件和它的每个附件都分开看成不同的文档
  - 如果邮件的附件是一个zip文件，我们想将附件解压缩并将其中的**每个文件都看成一个文档**

多格式/语言并存

- 与此不同的是，网络上的各种软件（如latex2html）有时把我们认为的**单一文档**（如一个PowerPoint文件或Latex文档）以幻灯片或者小节为单位切分成**多个HTML页面**，并将每个页面保存到独立的文件中
  - 这种情况下，我们反而希望将**多个文件合并成一个文档来构建索引**

多格式/语言并存

- 对于长文档而言，存在**“索引粒度”（indexing granularity）**的问题
  - 对于一个书库来说，将整本书作为索引单位通常是个糟糕的做法
  - 搜索“Chinese toys”，可能第1章提到“China”而最后一章提到了“toys”
  - 不符合需求
- 可取的做法是将**每章或每段**看成一个**微型文档**（mini-document）来建立索引
  - 此时匹配的结果可能与查询更相关
  - 而由于匹配单位的**粒度更小**，用户也更容易在文档中找到相关的段落

多格式/语言并存

- 为什么不进一步采用**更小粒度**的单位呢？比如将**每个句子作为索引单位**
- 存在**正确率和召回率之间的权衡问题**
  - 如果索引粒度太小，那么由于词项散布在多个细粒度文档中，我们就很可能错过那些重要的段落，也就是说此时**正确率高而召回率低**
  - 反之，如果索引粒度太大，我们就很可能找到很多不相关的匹配结果，即**正确率低而召回率高**

多格式/语言并存

- 如何确定索引的单位?
  - **文件**为单位?
  - 邮件为单位?
  - 如果邮件带有5个**附件**，怎么办?
  - 一组文件? (比如采用html格式写的某个PPT文档)

词条和词项

TOKENS AND TERMS

词条化

- 所谓**词条化**（tokenization）指的是将原始的字符流转换成一个个**词条（token）**的过程
- **词条化**将给定的字符序列拆分成一系列子序列的过程，其中每个子序列称为一个**词条（token）**
- 在非严格的情况下，词条往往和词项或词通用，然而，有时我们需要对词条和词条类进行严格的区分
  - **一个词条指的是在文档中出现的字符序列的一个实例**
  - **而一个词条类（type）指的是相同词条构成的集合**
- **一个词项指的是在信息检索系统词典中所包含的某个可能经过归一化处理的词条类**

词条化

- 实际的信息检索系统中，**词项往往和词条密切相关**
- 但是，词项未必就是原始的词条，实际上它往往要通过对**原始词条进行归一化**来得到
  - **例子：对“ to sleep perchance to dream”进行索引**
  - **5 个词条**
  - **4个词条类**
  - **3 个词项（将to看成停用词去掉）**
- 词条化的主要任务就是确定哪些才是正确的词条
  - 对于上例而言，该任务实现起来似乎非常简单，
  - 根据空格将字符串进行拆分并去掉标点符号即可
- BUT, 仅代表最简单情况，**真实情况并非如此**

词条化

- 实际上即使对于**单词之间存在空格的英文**来说也存在很多难以处理的问题
- 比如，英文中的**上撇号**“ ’ ” 既可以**代表所有关系也可以代表缩写**，应当在词条化过程中究竟应该如何对它进行处理?
- 例子：Mr. O’ Neill thinks that the boys’ stories about Chile’s capital aren’t amusing.
- 对其中的“ O’ Neill” 来说，**词条化的结果?**

neill
oneill
o’neill
o’ neill
o neill ?

词条化

- 不管是输入布尔查询或者自由文本查询，人们总是希望对文档和查询进行同样的词条化处理
- 采用相同的词条化工具，确保文本与查询中的同一字符串序列的处理结果相一致
- 不同语言下的词条化处理并不相同，处理时必须要知道文档的语言种类
- 大多数语言都有自己独特的特征模式
- 一种非常有效的语言种类识别（language identification）方法是利用短字符子序列作为特征来进行分类

词条化(Tokenization)

- **输入: “Friends, Romans and Countrymen”**
- **输出: 词条(Token)**
  - Friends
  - Romans
  - Countrymen
- 词条就是一个**字符串实例**
- 词条在经过进一步处理之后将放入倒排索引中的词典中
- 词条化中的问题
  - **词条如何界定?**

词条化

- **一系列问题:**
  - **Finland’s capital →**
  - **Finland? Finlands? Finland’s?**
  - **Hewlett-Packard → 看成Hewlett 和 Packard 两个词条?**

词条化

- state-of-the-art
- co-education
- San Francisco: 到底是一个还是两个词条?
  - 如何判断是一个词条?



Earth Syst. Sci. Data, 13, 4349–4383, 2021
https://doi.org/10.5194/essd-13-4349-2021
© Author(s) 2021. This work is distributed under
the Creative Commons Attribution 4.0 License.

Open Access Earth System Science Data

ERA5-Land: a state-of-the-art global reanalysis dataset for land applications

Joaquín Muñoz-Sabater[1], Emanuel Dutra[2,3], Anna Agustí-Panareda[1], Clément Albergel[4,5], Gabriele Arduini[1], Gianpaolo Balsamo[1], Souhail Boussetta[1], Margarita Choulga[1], Shaun Harrigan[1], Hans Hersbach[1], Brecht Martens[6], Diego G. Miralles[6], María Piles[7], Nemesio J. Rodríguez-Fernández[8], Ervin Zsoter[1], Carlo Buontempo[1], and Jean-Noël Thépaut[1]

[1]European Centre for Medium-range Weather Forecasts, Reading, UK
[2]Instituto Português do Mar e da Atmosfera, Lisbon, Portugal
[3]Instituto Dom Luiz, IDL, Faculty of Sciences, University of Lisbon, Lisbon, Portugal
[4]CNRM, Université de Toulouse, Météo-France, CNRS, Toulouse, France
[5]European Space Agency Climate Office, ECSAT, Didcot, UK
[6]Hydro-Climate Extremes Lab (H-CEL), Ghent University, Ghent, Belgium
[7]Image Processing Laboratory (IPL), Universitat de València, València, Spain
[8]Centre d'Etudes Spatiales de la Biosphère (CESBIO),
Université Toulouse 3, CNES, CNRS, INRAE, IRDe, Toulouse, France

Correspondence: Joaquín Muñoz-Sabater (joaquin.munoz@ecmwf.int)

Received: 9 March 2021 – Discussion started: 15 March 2021
Revised: 17 June 2021 – Accepted: 6 July 2021 – Published: 7 September 2021

词条化和短语查询

- **lowercase, lower-case, lower case ?**
- **词条化问题**往往也会和**短语查询**的处理相互影响
- 尤其是输入lowercase 或lower-case 或lower case 都想返回相同结果时影响更大
- **后两种输入情况**可以通过在连字符处拆分并对短语进行索引的方式来解决
- 但是要处理**第一种输入情况**就必须知道lowercase 有时可以写成两个词
- 并且系统也要基于这两个词来建立索引

词条化中数字的处理

- 3/20/91　　Mar. 12, 1991　　20/3/91
- 12/3/91
- 55 B.C.
- B-52　F-15
- MU5321　3U8729　HU7152 CA4305
- MU507
- G350　T8　K118　Z50　D1812

词条化中数字的处理

- **QCKNG-29MKJ-74G4B-X7DT8-JFHBB**
- 某种密钥：**324a3df234cb23e**
- **(800) 234-2333**
- **13800138000**
- **A4-BB-6D-64-3C-16**
- **210.41.224.162　192.168.1.1**

词条化中数字的处理

- 通常中间有空格
- 早期的IR系统可能不索引数字
  - 但是数字却常常很有用：比如在Web上查找错误代码
- 元数据是分开还是一起索引
  - 创建日期、格式等等

语言问题：法语和德语

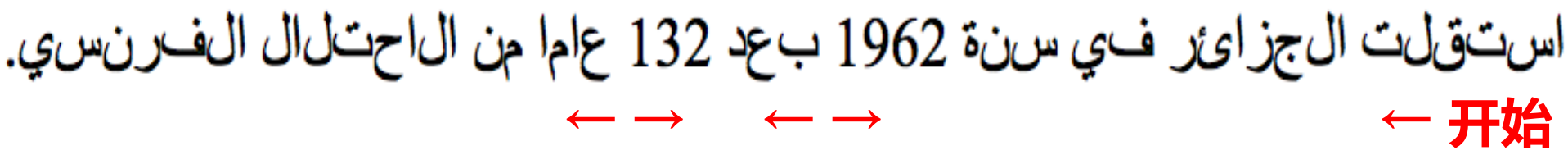
- 法语
  - L‘ensemble → 到底是一个还是两个词条?
    - L ? L’ ? Le ?
    - 但是常常希望 l’ensemble 能和un ensemble匹配
      - 至少在2003年以前，Google没有这样处理
        - 国际化问题!

语言问题：法语和德语

- **德语--复合名词连写**
- **Lebensversicherungsgesellschaftsangestellter**
  - **‘life insurance company employee’**
  - **德语检索系统往往要使用一个复合词拆分的模块，而且该模块对检索结果的提高有很大帮助(可以提高15%)**
- **die Bayerischen Motorenwerke**
  - **(Bavarian Motor Works)**
  - **宝马（BMW），巴伐利亚发动机制造厂股份有限公司**

语言问题：阿拉伯文

- **阿拉伯文 (或希伯来文)** 通常从右到左书写，但是某些部分(如数字)是从左到右书写
- 词之间是分开的，但是单词中的字母形式会构成复杂的连接方式



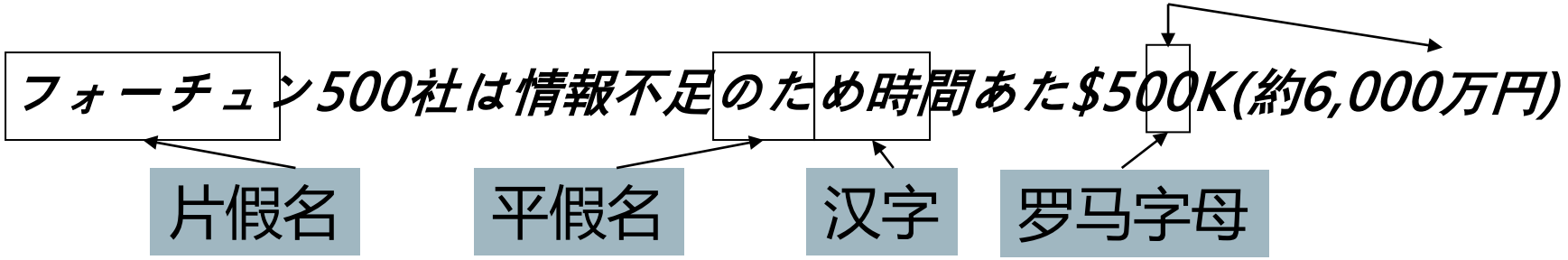
- 'Algeria achieved its independence in 1962 after 132 years of French occupation.'
- **在Unicode编码方式下，表面的表示方式很复杂，但是存储上倒是十分直接**

语言问题：中文和日文

- **中文和日文词之间没有间隔**
  - 莎拉波娃现在居住在美国东南部的佛罗里达。今年4月9日，莎拉波娃在美国第一大城市纽约度过了18岁生日。生日派对上，莎拉波娃露出了甜美的微笑。
- **分词结果无法保证百分百正确**
  - **“和尚”**
  - monk
  - and, still, respect

语言问题：中文和日文

- **日文中可以同时使用多种类型的字母表**
  - **日期/数字可以采用不同的格式**



而终端用户可能完全用平假名方式输入查询！

中文分词(Chinese Word Segmentation)

- **对于中文，分词的作用实际上是要找出一个个的索引单位**
- **例子：李明天天都准时上班**
- **索引单位**
  - **字**：李 明 天 天 都 准 时 上 班
    - 索引量太大，**查全率**百分百，但是**查准率**低，比如查“明天” 这句话也会出来
  - **词**：李明 天天 都 准时 上班
    - 索引量大大降低，**查准率较高，查全率不是百分百**，而且还会受分词错误的影响，比如上面可能会切分成：李 明天 天都 准时 上班

分词的方法

- 另一个解决方法则摒弃了基于词的索引策略而采用**短字符序列**（如字符的***k*-gram** 方法）。这种方法并不关心词项是否会跨越词的边界
- 该方法之所以能够引起人们的兴趣主要有以下3个原因
  - **第一，一个汉字更像是一个音节而不是字符，它往往具有语义信息**
  - **第二，大部分词都很短（最常见的汉语词长度是2 个字）**
  - **第三，由于缺乏公认的分词标准，词的边界有时也很难确定**
- 字词混合方式/**k-gram**/多k-gram混合 (一般原则，没把握的情况下**细粒度优先**)
- 即使对英语来说，有些情况下**词边界的判断**也缺乏标准，很多也只是习惯用法而已，比如notwithstanding 和 not to mention、into 和on to中的词边界标准就不一样

分词的方法

- **基于词典的最大匹配法**（采用**启发式规则**来进行未定义词识别）
- **基于机器学习序列模型的方法**（如**隐马尔可夫**模型或**条件随机场**模型）
  - 由于**存在多种切分**，上述分词方法都有可能导致错误的切分结果
  - 因此，永远不能保证只能得到一个**完全一致的唯一切分结果**

中文分词和检索

- 以下是当前某些研究的结论或猜测，仅供参考

- **并非分词精度高一定检索精度高**
  - 评价标准不同
  - 分词规范问题：鸡蛋、鸭蛋、鹌鹑蛋……
  - 目标不同
- **检索中的分词**
  - 查询和文档切分采用一致的分词系统
  - 速度快
  - 倾向细粒度，保证召回率
  - 多粒度并存
- **搜索引擎中的分词方法**
  - 猜想：大词典+统计+启发式规则

停用词

- **停用词（stop word）**
- 一些常见词在文档和用户需求进行匹配时价值并不大，需要彻底从词汇表中去除
- 一个常用的**生成停用词表**的方法
  - 将词项按照文档集频率（collection frequency，每个词项在文档集中出现的频率）从高到低排列
  - **手工选择**那些语义内容与文档主题关系不大的高频词作为停用词
  - 停用词表中的每个词将在索引过程中被忽略

停用词

- 根据停用词表(stop list), 将那些最常见的词从词典中去掉。比如直观上可以去掉**一般不包含语义信息的词**
  - **the, a, and, to, be**
  - **“的”、“得”、“地”等**
  - **口语**：然后、那个
  - **高频词**：前30个词就占了 ~30% 的倒排记录表空间

停用词

a	an	and	are	as	at	be	by	for	from
has	he	in	is	it	its	of	on	that	the
to	was	were	will	with					

Reuters-RCV1 语料库中的25个停用词

停用词

Nevertheless, 现代信息检索系统中倾向于不去掉停用词

- 在保留停用词的情况下，采用良好的压缩技术(第五章)后，停用词所占用的空间可以大大压缩，最终它们在整个倒排记录表中所占的空间比例很小
- Web搜索引擎通常都不用停用词表
- 一些现代IR系统更关注如何利用语言的统计特性来更好地处理常见词问题
- 采用良好的查询优化技术基本不会增加查询处理的开销

信息检索系统不断发展的历程

- 大停用词表（200～300 个词）→小停用词表（7～12个词）→不用停用词

停用词

- **所谓的停用词并不一定没用**，such as：
  - 短语查询: “King of Denmark”
  - 歌曲名或者台词:“Let it be”，“To be or not to be”
  - “关系型” 查询: “flights to London”

词条归一化(Normalization)成词项

- 将文档和查询中的词**归一化**成同一形式
  - U.S.A. 和 USA
- **词条归一化**（token normalization）
  - 将看起来不完全一致的多个词条归纳成一个等价类，以便在它们之间进行匹配的过程
- 最常规的做法是隐式地建立等价类，每类可以用其中的某个元素来命名

词条归一化(Normalization)成词项

- **归一化的结果就是词项，而词项就是我们最终要索引的对象**
- 可以采用**隐式规则**的方法来表示多个词条可以归一成同一词项，比如
  - 剔除句点
    - U.S.A., USA
  - 剔除连接符
    - anti-discriminatory, antidiscriminatory

归一化中的语言问题

- **重音符**: 如法语中 résumé vs. resume
- 日耳曼语系中的**元音变化**: 如德语中的 Tubingen vs. Tübingen
  - 应该是一致的
- 最重要的准则
  - 用户在输入查询时遇到这些词如何输入?

- 即使在有重音符号的语言中，用户也往往不输入这些符号
  - 常常归一化成不带重音符号的形式
    - Tubingen, Tübingen → Tubingen

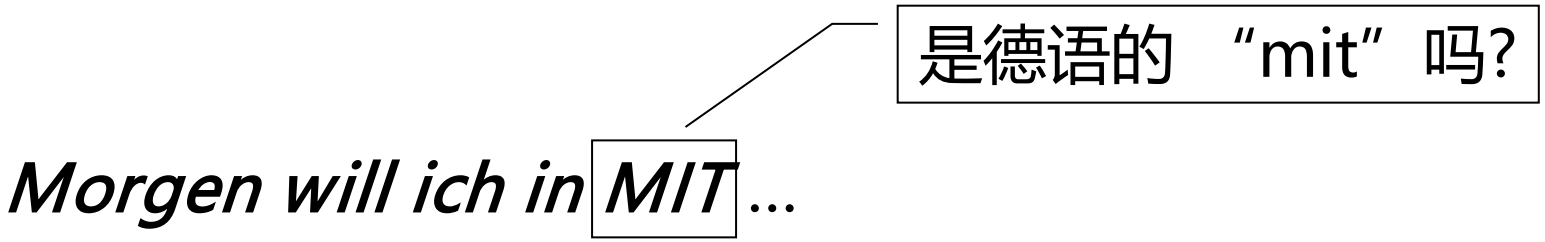
归一化中的语言问题

- **时间格式**
  - 7月30日 vs. 7/30
- **日语中用假名或者汉字表示日期**

星期一：月曜日（げつようび）
星期二：火曜日（かようび）
星期三：水曜日（すいようび）
星期四：木曜日（もくようび）
星期五：金曜日（きんようび）
星期六：土曜日（どようび）
星期日：日曜日（にちようび）

归一化中的语言问题

- 词条化和归一化都可能与语言相关，因此必须要做**语言识别**
- 另外，谨记要将文档和查询中的**同义词**归一化成同一形式



提纲

1. 文档
2. 词项
    1. 通常做法+非英语处理
    2. 英语
3. 跳表指针
4. 短语查询

大小写问题

- **Case-folding**
- **可以将所有字母转换成小写形式**
  - **例外: 句中的大写单词?**
    - e.g., General Motors（GM，通用公司）
    - Game Master?
    - FED (美联储)vs. fed(饲养)
    - SAIL (印度钢铁管理局) vs. sail(航行)
  - 通常情况下将所有字母转成小写是一种很合适的方式
  - 因为**用户倾向于用小写方式输入**



大小写问题

- **Google的例子:**
  - **查询 C.A.T.**
  - **排名第一的结果是“cat”而不是 Caterpillar Inc.**
- **Baidu**



归一化成词项

- 除了前面互换方式(即能够归一化成同一词项的词条之间完全平等，可以互换)之外，另一种方式是**非对称扩展 (asymmetric expansion)**
- 一个非对称扩展更适合的例子
  - **输入: window　　搜索: window, windows**
  - **输入: windows　　搜索: Windows, windows, window**
  - **输入: Windows　　搜索: Windows**
  - 为什么反过来不行?
- 这种方法可能更强大，但是效率低一些

番外：**软件用语的大小写规范**

Windows, Java；缩写 XML, Spring Boot等

同义词词典(Thesauri)及soundex方法

- **同义词和同音/同形异义词的处理**
  - **E.g., 手动建立词典，记录这些词对**
    - **car = automobile　　color = colour**
    - **fridge = refrigerator**
  - **利用上述词典进行索引**
    - **当文档包含 automobile时, 利用car-automobile进行索引**
  - **或者对查询进行扩展**
    - **当查询包含 automobile时，同时也查car**

同义词词典(Thesauri)及soundex方法

- **拼写错误的处理**(Clinton→Klinten)
  - **soundex方法，基于发音建立词之间的关系(一种解决方法)**
    - 返回由四个字符组成的代码
    - 评估两个字符串在发音时的相似性
    - 参考3.4节

词干还原（Stemming）和词形归并(Lemmatization)

- 将单词的**屈折变体形式还原为原形**
  - am, are, is → be
  - car, cars, car's, cars' → car
  - the boy's cars are different colors → the boy car be different color
- 词性归并意味中将单词的变形形式“适当”还原成一般词典中的单词形式
  - found → find? found?

词干还原（Stemming）和词形归并(Lemmatization)

- **将词项归约(reduce)成其词干(stem)，然后再索引**
- **“词干还原” 意味着词缀的截除**
  - 与语言相关
  - 比如，将 automate(s), automatic, automation都还原成 automat

for example compressed and compression are both accepted as equivalent to compress.



for exampl compress and compress ar both accept as equival to compress

词干还原（Stemming）和词形归并(Lemmatization)

- 两个概念意义不同
- **词干还原（stemming）**
  - 通常指的是一个很粗略的去除单词两端词缀的启发式过程，并且希望大部分时间它都能达到这个正确目的，这个过程也常常包括去除派生词缀
- **词形归并**
  - 通常指利用词汇表和词形分析来去除屈折词缀，从而返回词的原形或词典中的词的过程，返回的结果称为**词元（lemma）**

- 例：给定词条saw
  - 词干还原过程可能仅返回s，而词形归并过程将返回see或者saw
  - 具体返回哪个词取决于在当前上下文中saw到底是动词还是名词
- **词干还原**在一般情况下会**将多个派生相关词合并在一起**
- **词形归并**通常**只将同一词元的不同屈折形式进行合并**

Porter算法

- **英语词干还原中最常用的算法**
  - 结果表明该方法不差于其他的词干还原方法
- 一些规定+ 5 步骤的归约过程
  - 步骤有先后顺序
  - 每一步都包含一系列命令
- 一些规定，比如: 选择可应用规则组中包含最长词缀的规则
  - SSES →SS caresses →caress
  - S → cats →cat

Porter中的典型规则

- ies → i
- ational → ate
- tional → tion
- **规则适用条件的表达**
  - (m>1) EMENT →
    - replacement → replac
    - cement → cement
  - *m* > 1 表示除去EMENT 之外的单词部分的长度要大于1

Martin Porter

- 2000年度 Tony Kent Strix award得主
  - 信息检索领域一个著名的奖项
- Porter’s stemmer，有很多语言的版本



其他词干还原工具(stemmer)

- Snowball工具
  - 支持多种语言的stemming(法语、德语、葡萄牙语、西班牙语、挪威语等等)
- Lovins
  - http://www.comp.lancs.ac.uk/computing/research/stemming/general/lovins.htm
  - 单遍扫描，最长词缀剔除 (大概250条规则)
  - 全部基于词形分析 – 对于检索来说最多只能提供一定的帮助(at most modest benefits for retrieval)

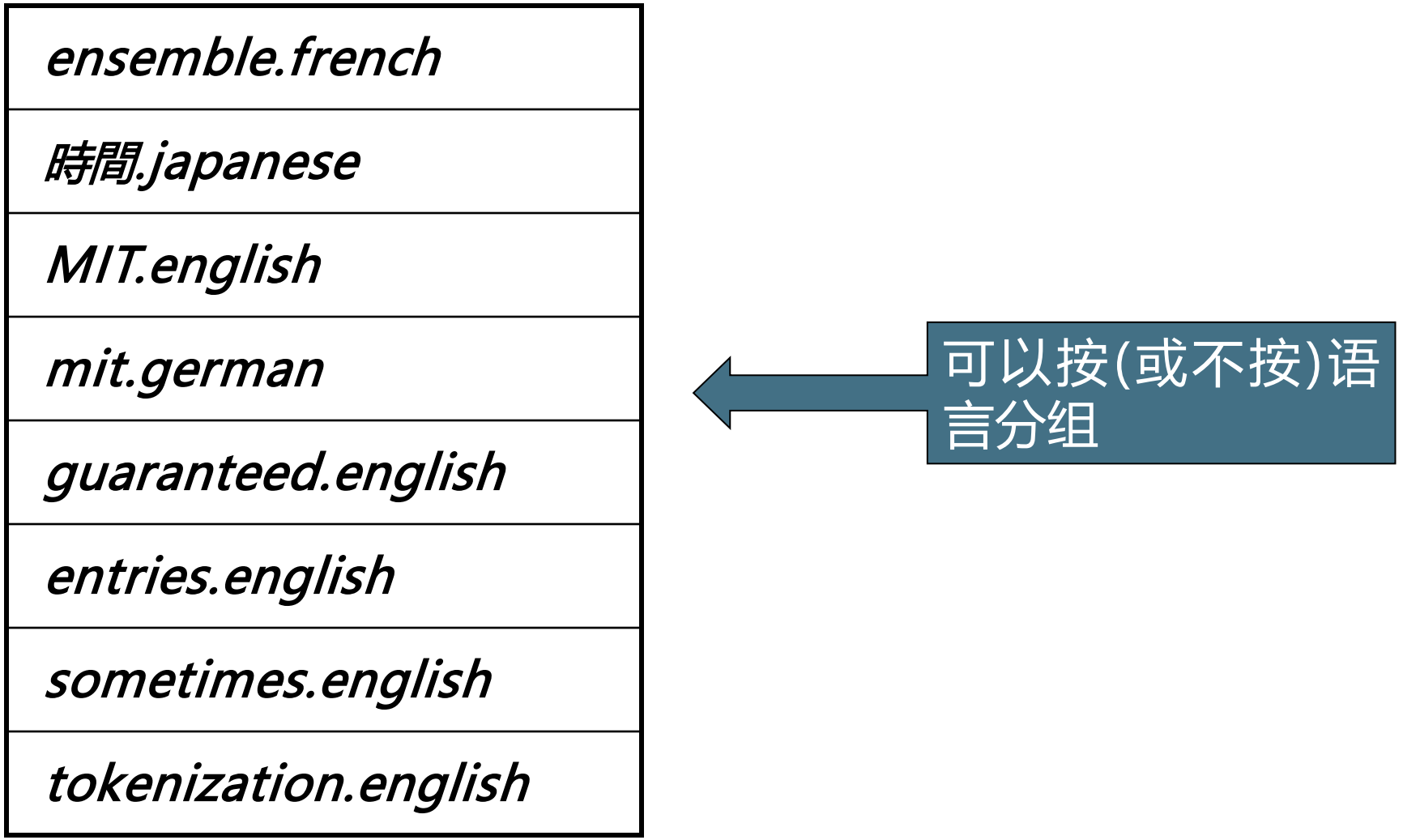
其他词干还原工具(stemmer)

- **词干还原及其它归一化工作对检索的帮助**
  - **英语： 结果要一分为二，对某些查询来说提高了召回率，但是对另外一些查询来说降低了正确率**
  - 比如, **operative**
  - **operate operating operates operation operative operatives operational**
  - **→ oper**
  - 对西班牙语、德语、芬兰语等语言非常有用
  - 其中对于芬兰语有30%的性能提高!

语言特性

- 上述很多转换处理具体实现时
  - 都与**语言本身**有关
  - 常常和具体应用有关
- 上述过程可以插件方式植入索引过程
- 存在很多开源和商业插件可用

词典入口示意图


ensemble.french
時間.japanese
MIT.english
mit.german
guaranteed.english
entries.english
sometimes.english
tokenization.english
可以按(或不按)语言分组

Review

- **TF**
- **DF**
- **列举停用词**
- **‘life insurance company employee’**
- **3-gram表示**
- **词干还原**

练习

正确率(Precision) 返回结果文档中正确的比例

召回率(Recall) 全部相关文档中被返回的比例

- **请判断如下说法是否正确。**

a. 在布尔检索系统中，进行词干还原从不降低正确率。 ✗

b. 在布尔检索系统中，进行词干还原从不降低召回率。 ✓

c. 词干还原会增加词项词典的大小。 ✗

d. 词干还原应该在构建索引时调用，而不应在查询处理时调用。 ✗

Review

1. **文档**
2. **词项**
    1. 通常做法+非英语处理
    2. 英语
3. **跳表指针**
4. **短语查询**

文档分析

- **文档格式处理**
  - pdf/word/excel/html?
- **文档语言识别**
  - 英文、法语、阿拉伯文、日语、中文等
- **文档编码识别**
  - ASCII、UTF-8等

词项集合的确定

- **词条化**
- **去除停用词**
- **词项归一化**
- **词干还原和词形归并**

词条化

- 所谓**词条化**（tokenization）指的是将原始的字符流转换成一个个**词条（token）**的过程
- **词条化**将给定的字符序列拆分成一系列子序列的过程，其中每个子序列称为一个**词条（token）**
- 在非严格的情况下，词条往往和词项或词通用，然而，有时我们需要对词条和词条类进行严格的区分
  - 一个**词条**指的是在文档中出现的字符序列的一个实例
  - 而一个**词条类**（type）指的是相同词条构成的集合
- 一个**词项**指的是在信息检索系统词典中所包含的某个可能经过归一化处理的词条类

词条化(Tokenization)

- 输入: “Friends, Romans and Countrymen”
- 输出: 词条(Token)
  - Friends
  - Romans
  - Countrymen
- 词条 就是一个**字符串实例**
- 词条在经过进一步处理之后将放入倒排索引中的词典中
- 词条化中的问题
  - **词条如何界定?**

- **词条化和短语查询**
- **词条化中数字的处理**
- **语言问题：**
  - 法语和德语
  - 阿拉伯文
  - 中文和日文
  - 英文

分词的方法

- 另一个解决方法则摒弃了基于词的索引策略而采用短字符序列（如字符的***k*-gram** 方法）。这种方法并不关心词项是否会跨越词的边界
- 该方法之所以能够引起人们的兴趣主要有以下3个原因
  - **第一，一个汉字更像是一个音节而不是字符，它往往具有语义信息**
  - **第二，大部分词都很短（最常见的汉语词长度是2 个字）**
  - **第三，由于缺乏公认的分词标准，词的边界有时也很难确定**
- 字词混合方式/k-gram/多k-gram混合 (一般原则，没把握的情况下**细粒度优先**)
- 即使对英语来说，有些情况下词边界的判断也缺乏标准，很多也只是习惯用法而已，比如notwithstanding 和 not to mention、into 和on to中的词边界标准就不一样

停用词

- 根据停用词表(stop list), 将那些最常见的词从词典中去掉。比如直观上可以去掉**一般不包含语义信息的词**
  - **the, a, and, to, be**
  - **“的”、“得”、“地”等**
  - **口语**：然后、那个
  - **高频词**：前30个词就占了 ~30% 的倒排记录表空间

停用词

Nevertheless, 现代信息检索系统中倾向于**不去掉停用词**

- 在保留停用词的情况下，采用良好的**压缩技术**(第五章)后，停用词所占用的空间可以大大压缩，最终它们在整个倒排记录表中所占的空间比例很小
- Web搜索引擎通常都不用停用词表
- 一些现代IR系统更关注如何利用语言的统计特性来更好地处理常见词问题
- 采用良好的**查询优化技术**基本不会增加查询处理的开销

信息检索系统不断发展的历程

- 大停用词表（200～300 个词）→小停用词表（7～12个词）→不用停用词

词条归一化(Normalization)成词项

- **归一化的结果就是词项，而词项就是我们最终要索引的对象**
- 可以采用**隐式规则**的方法来表示多个词条可以归一成同一词项，比如
  - 剔除句点
    - U.S.A., USA
  - 剔除连接符
    - anti-discriminatory, antidiscriminatory
- **重音符**: 如法语中 résumé vs. resume
- 日耳曼语系中的**元音变化**: 如德语中的 Tubingen vs. Tübingen

大小写问题

- **Case-folding**
- **可以将所有字母转换成小写形式**
  - **例外: 句中的大写单词?**
    - e.g., General Motors（GM，通用公司）
    - Game Master?
    - FED (美联储)vs. fed(饲养)
    - SAIL (印度钢铁管理局) vs. sail(航行)
  - **通常情况下将所有字母转成小写是一种很合适的方式**
  - **因为用户倾向于用小写方式输入**



同义词词典(Thesauri)及soundex方法

- **同义词和同音/同形异义词的处理**
  - E.g., 手动建立词典，记录这些词对
    - car = automobile　　color = colour
    - fridge = refrigerator
  - 利用上述词典进行索引
    - 当文档包含 automobile时, 利用car-automobile进行索引
- **拼写错误的处理**(Clinton→Klinten)
  - soundex方法，基于发音建立词之间的关系(一种解决方法)

词干还原（Stemming）和词形归并(Lemmatization)

- **将词项归约(reduce)成其词干(stem)，然后再索引**
- **“词干还原” 意味着词缀的截除**
  - 与语言相关
  - 比如，将 automate(s), automatic, automation都还原成 automat

for example compressed and compression are both accepted as equivalent to compress.



for exampl compress and compress ar both accept as equival to compress

词干还原（Stemming）和词形归并(Lemmatization)

- 两个概念意义不同
- **词干还原（stemming）**
  - 通常指的是一个很粗略的去除单词两端词缀的启发式过程，并且希望大部分时间它都能达到这个正确目的，这个过程也常常包括去除派生词缀
- **词形归并**
  - 通常指利用词汇表和词形分析来去除屈折词缀，从而返回词的原形或词典中的词的过程，返回的结果称为**词元（lemma）**

Porter中的典型规则

- ies → i
- ational → ate
- tional → tion
- **规则适用条件的表达**
  - (m>1) EMENT →
    - replacement → replac
    - cement → cement
  - *m* > 1 表示除去EMENT 之外的单词部分的长度要大于1

提纲

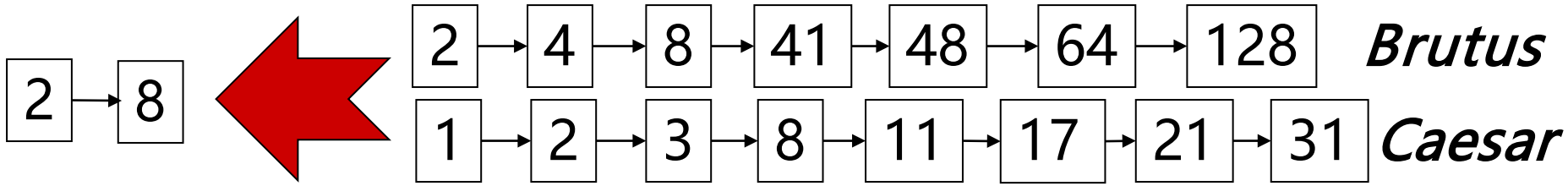
1. 文档
2. 词项
    1. 通常做法+非英语处理
    2. 英语
3. 跳表指针
4. 短语查询

快速倒排表合并—跳表法

FASTER POSTINGS MERGES: SKIP POINTERS/SKIP LISTS

基本合并算法的回顾

- **两个指针，同步扫描，线性时间**

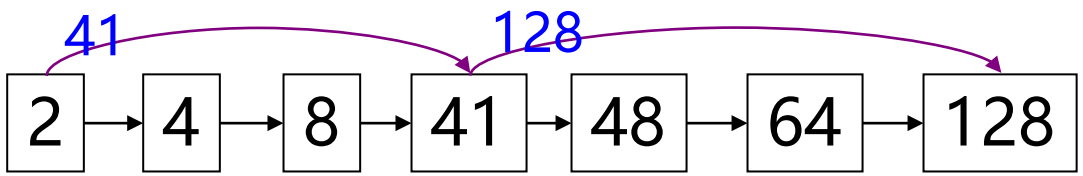


两个表长度为m和n的话，上述合并时间复杂度为 O($m+n$)

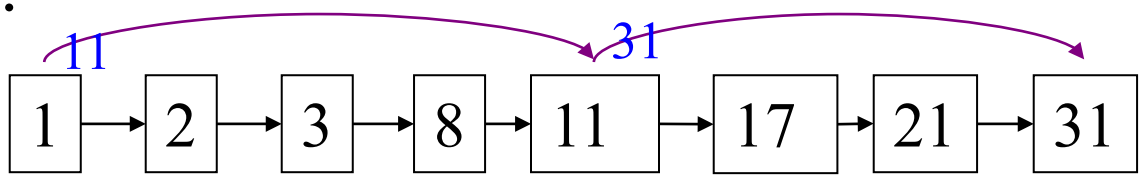
能否做得更好？答案是可以(**如果索引不常变化的话**)

索引构建时为倒排记录表增加跳表指针

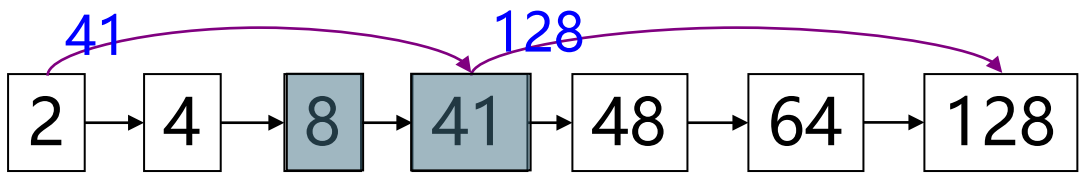
- 一种实现的方法是采用**跳表（skip list）**，在构建索引的**同时在倒排记录表上建立跳表**
- 为什么可以加快速度?
  - 可以跳过那些不可能的检索结果

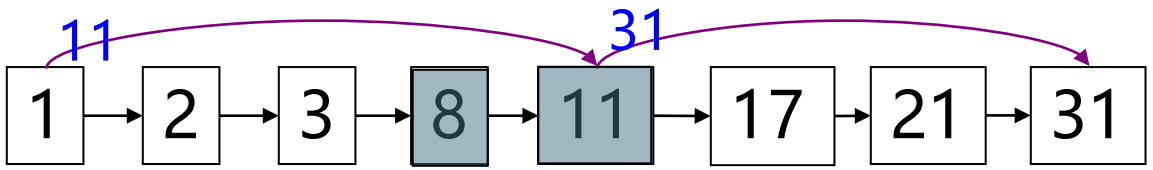


- 如何做? 也就是在什么地方加跳表指针?



基于跳表指针的查询处理

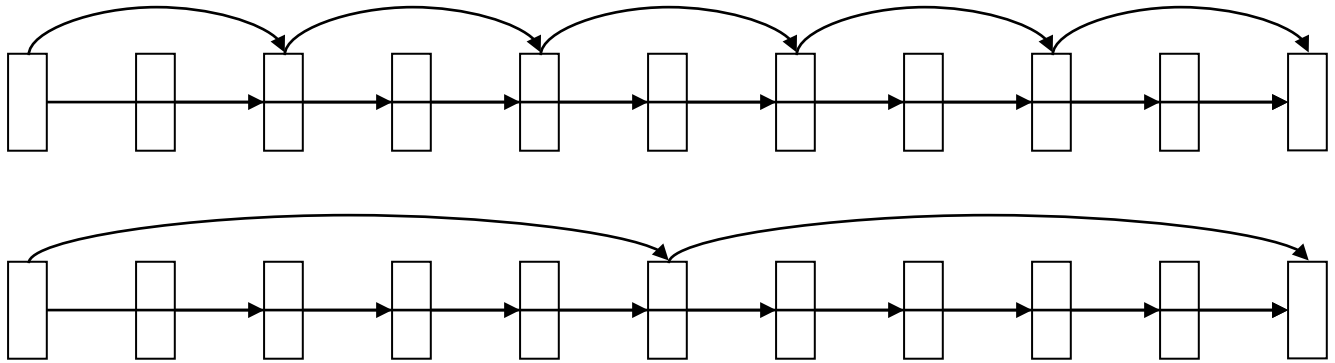




- 假定匹配到上下的指针都指向8，接下来两个指针都向下移动一位
- **比较41和11**，11小
- 此时看11上面的跳表指针，指向31，31仍然比41小，于是下指针可以直接跳过中间的11、17、21、31

跳表指针的位置

- **指针数目过多过少都不合适，要有一个均衡性**
  - **指针越多→ 跳步越短 ⇒ 更容易跳转，但是需要更多的与跳表指针指向记录的比较**
  - **指针越少→ 比较次数越少，但是跳步越长 ⇒ 成功跳转的次数少**



跳表指针的位置

- **简单的启发式策略**
- **对于长度为L的倒排记录表，每 $\sqrt{L}$ 处放一个跳表指针，即均匀放置**
  - 均匀放置方法**忽略了查询词项的分布情况**
  - **假设考虑某个词项在文档有集中分布，则调整指针位置**

基于跳表指针的倒排记录表合并算法

```
INTERSECTWITHSKIPS(p1, p2)
 1  answer ← ⟨ ⟩
 2  while p1 ≠ NIL and p2 ≠ NIL
 3  do if docID(p1) = docID(p2)
 4        then ADD(answer, docID(p1))
 5             p1 ← next(p1)
 6             p2 ← next(p2)
 7        else if docID(p1) < docID(p2)
 8                then if hasSkip(p1) and (docID(skip(p1)) ⩽ docID(p2))
 9                     then while hasSkip(p1) and (docID(skip(p1)) ⩽ docID(p2))
10                          do p1 ← skip(p1)
11                     else p1 ← next(p1)
12                else if hasSkip(p2) and (docID(skip(p2)) ⩽ docID(p1))
13                     then while hasSkip(p2) and (docID(skip(p2)) ⩽ docID(p1))
14                          do p2 ← skip(p2)
15                     else p2 ← next(p2)
16  return answer
```

跳表指针的位置

- 如果**索引相对静态**，均匀方式方法是一种很简便的方法，但是如果**索引经常更新**造成L经常变化，均匀方式方法就很不方便
- **跳表方式在过去肯定是有用的，但是对于现代的硬件设备而言，如果合并的倒排记录表不能全部放入内存的话，上述方式不一定有用 (Bahle et al. 2002)**
- **现代计算机的CPU速度很快，但是磁盘访问速度仍然很慢**
- **更大的倒排记录表(含跳表)的I/O开销可能远远超过内存中合并带来的好处（如何减少磁盘上倒排记录表的存储空间）**

提纲

1. 文档
2. 词项
    1. 通常做法+非英语处理
    2. 英语
3. 跳表指针
4. 短语查询

短语查询及位置索引

PHRASE QUERIES AND POSITIONAL INDEXES

短语查询

- **输入查询作为一个短语整体，比如 “Stanford university” “中国科学院”**
- **因此，句子 “I went to university at Stanford” 就不应该是答案 （“我去了中国 农业科学院”）**
  - **有证据表明，用户很容易理解短语查询的概念，这也是很多搜索引擎”高级搜索”中比较成功的一个功能**
  - **但是很多查询是隐式短语查询， information retrieval textbook → [information retrieval] textbook**
- **这种情况下，倒排索引仅仅采用如下方式是不够的**
- **term + docIDs**

第一种做法: 双词(Biword)索引

- Bi-
  - Binary
  - Binocular、Bilingual
  - Biweekly、Biannual
- 每两个连续的词组成词对(作为短语)来索引
- 比如文本片段 “Friends, Romans, Countrymen” 会产生两个词对
  - friends romans
  - romans countrymen
- 索引构建时，将每个词对看成一个词项放到词典中，直接处理两个词组成的短语查询

更长的短语查询处理

- 例子： stanford university palo alto
- 处理方法：将其拆分成基于双词的布尔查询式
  - stanford university AND university palo AND palo alto
- 2-gram(K-gram)
  - 一个K-gram代表由K个字符组成的序列
- 如果不检查文档，无法确认满足上述表达式的文档是否真正满足上述短语查询。也就是说满足上述布尔表达式只是满足短语查询的充分条件

扩展的双词（Extended Biword）

- **对待索引文档进行词性标注**
- **将词项进行组块，每个组块包含名词 (N) 和冠词/介词 (X)**
- **称具有NX*N形式的词项序列为扩展双词**(extended biword)
  - 将这样扩展词对作为词项放入词典中
- 例子：catcher in the rye (书名： 麦田守望者)
  - N X X N
- **查询处理：将查询也分析成N和X序列**
  - 将查询切分成扩展双词
  - 在索引中查找: catcher rye

关于双词索引

- **二元词索引的概念可以扩展到更长的词序列**
- **Nevertheless，词典中词项数目剧增，导致索引空间也激增**
- 如果3词索引，那么更是空间巨大，无法忍受
- 如果索引中包含**变长的词序列**，通常就称为**短语索引（phrase index）**

关于双词索引

- **实际上，利用二元词索引来处理单个词的查询不太方便（必须要扫描整个词汇表来发现包含该查询词的二元词），因此同时还需要有基于单个词的索引**
- 尽管总有可能得到错误的匹配结果，但是在**长度为3或者更长的索引短语**上发生匹配错误的可能性实际上却很小
- 然而在另一方面，**存储更长的短语很可能会大大增加词汇表的大小**
  - 长度超过2的短语并维护其索引令人生畏
- **双词索引方法并不是一个标准的做法（即倒排索引中一般不会全部采用双词索引方法），但是可以和其他方法混合使用**

第二种做法: 带位置信息索引 (Positional indexes)

- **在倒排记录表中，对每个term在每篇文档中的每个位置(偏移或者单词序号)进行存储:**
  - **<term, 出现term的文档篇数;**
  - **doc1: 位置1, 位置2 … ;**
  - **doc2: 位置1, 位置2 … ;**
  - **等等>**

位置索引的例子

- **对于输入的短语查询，需要在文档的层次上进行迭代(不同位置上)合并**
- **不仅仅简单合并，还要考虑位置匹配**

<*be*: 993427;

1: 7, 18, 33, 72, 86, 231;

2: 3, 149;

4: 17, 191, 291, 430, 434;

5: 363, 367, ...>

1,2,4,5这几篇文章中哪篇包含“*to be or not to be*”？

短语查询的处理

- 短语查询： “to be or not to be”
- **对每个词项，抽出其对应的倒排记录表**: to, be, or, not.
- 合并<docID:位置 >表，考虑“to be or not to be”

- **to:**
  - **2:1,17,74,222,551;**
  - **4:8,16,190,429,433;**
  - **7:13,23,191; ...**
- **be:**
  - **1:17,19;**
  - **4:17,191,291,430,434;**
  - **5:14,19,101; ...**

邻近式查询(Proximity query)

- **邻近搜索中的搜索策略与此类似，不同的是此时考虑前后位置之间的距离不大于某个值**
- **/k**
- **LIMIT! /3 STATUTE /3 FEDERAL /2 TORT**
  - **/k表示 “在k个词之内”**
- **很明显，位置索引可以处理邻近式查询，而双词索引却不能**

位置索引的大小

- **位置索引增加了位置信息，因此空间较大，但是可以采用索引压缩技术进行处理(参见第五章)**
- 当然，相对于没有位置信息的索引，**位置索引的存储空间**明显大于无位置信息的索引
- 位置索引目前是实际检索系统的**标配**，这是因为实际中用户需要处理**短语(显式和隐式)和邻近式查询**

位置索引的大小

- **词项在每篇文档中的每次出现都需要一个存储单元**
- **因此索引的大小依赖于文档的平均长度**
  - 平均Web页面的长度<1000 个词项
  - 美国证监会文件(SEC filings), 书籍, 甚至一些史诗… 很容易就超过 100,000 个词项

文档大小	倒排记录表的数目	位置索引存储单元
1000	1	1
100,000	1	100

- 假定某个词项的出现频率是0.1%
- 大文件条件下会造成索引存储空间大小的**两个数量级**规模的增长

一些经验规律

- **位置索引的大小大概是无位置信息索引的2-4倍**
- **位置索引大概是原始文本容量的35-50%**
- 提醒
  - 上述经验规律适用于**英语及类英语**的语言

混合索引

- 上述两种索引方式(双词索引、位置索引)可以混合使用
  - 处理开销最大的短语查询往往是这样一些短语：每个词都非常常见，但是组合起来却相对很少见
  - 某些特定的短语采用位置索引的方式那么效率不高
  - “Michael Jackson”，“Britney Spears”
  - “Air Jordan”，“Macbook Air”
  - “The Who” （英国一著名摇滚乐队），采用位置索引，效率更低

混合索引

- Williams et al. (2004)对一种混合的索引机制进行了评估
  - 采用混合机制，那么对于典型的Web查询(比例)来说，相对于只使用位置索引而言，空间开销增加了26%，而减少了3/4的完成时间

本讲小结

- 索引构建过程(特别是预处理)
- 如何对索引文档进行处理来得到词典
  - 理解文档(document)的概念
  - 词条化(Tokenization)，理解词条(token)的概念
  - 词项生成，理解词项(term)的概念
- 倒排记录表
  - 更快的合并算法: 跳表法(skip list)
  - 短语查询的处理及带位置信息的倒排索引

参考资料

- 《信息检索导论》第2章
- MG 3.6, 4.3; MIR 7.2
- Porter’s stemmer: http://www.tartarus.org/~martin/PorterStemmer/
- 跳表理论: Pugh (1990)
  - Multilevel skip lists give same O(log n) efficiency as trees
- H.E. Williams, J. Zobel, and D. Bahle. 2004. “Fast Phrase Querying with Combined Indexes”, ACM Transactions on Information Systems.
- http://www.seg.rmit.edu.au/research/research.php?author=4
- D. Bahle, H. Williams, and J. Zobel. Efficient phrase querying with an auxiliary index. SIGIR 2002, pp. 215-221.